

The perceptron: notes

Thomas Serre · Professor of Cognitive & Psychological Sciences · Carney Institute for Brain Science, Brown University · Thursday, October 8 · Fall 2026

The perceptron is the first model neuron that learned its weights from labelled examples. Rosenblatt introduced it in 1958 [3] and built it in hardware: the Mark I Perceptron had a 20×20 grid of photocells as input and motor-driven potentiometers as weights. Minsky and Papert's book *Perceptrons* [1] proved that a single layer cannot compute XOR and other functions like it, and interest and funding for neural networks fell for more than a decade. The perceptron is small enough to run by hand, and it already does what later learning rules do: compare the output with the label, and change the weights in proportion to the input.

The model

The neuron has one weight per input. It computes the weighted sum $w^\top x$ of its input x , a dot product, and outputs its sign:

$$\hat{y} = \text{step}(w^\top x) = \begin{cases} +1 & \text{if } w^\top x > 0, \\ -1 & \text{if } w^\top x < 0. \end{cases}$$

Labels are coded $y = +1$ and $y = -1$, as in the lecture. The *decision boundary* is the set of inputs with $w^\top x = 0$. In two dimensions it is a line. This perceptron has no bias, so the line passes through the origin. The weight vector w is perpendicular to the line: a point x on the line has $w^\top x = 0$, which means x is at right angles to w . The weight vector points into the $+1$ side. A point exactly on the line ($w^\top x = 0$) gets no class and counts as a mistake. A bias b is added back by giving the neuron an extra input fixed at 1, with weight b , so that $(w, b)^\top (x, 1) = w^\top x + b$, and the rule below learns b like any other weight.

The rule

Present the labelled examples one at a time. If the neuron is right ($y w^\top x > 0$), nothing changes. If it is wrong,

$$w \leftarrow w + \eta y x,$$

where $\eta > 0$ is the step size, the *learning rate*. A missed $+1$ example is added to w , and a false alarm on a -1 example is subtracted from it. The rule is often written $w \leftarrow$

$\mathbf{w} + \eta(y - \hat{y})\mathbf{x}$. With labels ± 1 , $y - \hat{y}$ is 0 when the neuron is right and $2y$ when it is wrong, so the two forms differ only by a factor of 2 in η .

Why does this help? After the update, the weighted sum for the same example is $(\mathbf{w} + \eta y \mathbf{x})^\top \mathbf{x} = \mathbf{w}^\top \mathbf{x} + \eta y \|\mathbf{x}\|^2$. Since $\|\mathbf{x}\|^2 > 0$, the sum moves toward the sign of the label: up for a +1 example, down for a -1 example. The line turns toward the missed point, and past it if the step is large enough. One update can push another example to the wrong side, which is why the examples are presented again, pass after pass, until a full pass makes no mistake.

Two runs by hand

Four points in the plane, A = (-3, 1) and B = (1, -2) with $y = -1$, C = (-1, 3) and D = (-2, 3) with $y = +1$, are presented in the order A, B, C, D on every pass, with $\eta = 1$. Each row of a table is one presentation: $\mathbf{w}^\top \mathbf{x}$ uses the weights of the row above, and the last column gives the weights after the step.

Run 1: start from $\mathbf{w} = (0, 0)$. The first example is always a mistake, because every weighted sum is 0.

Pass	Example	y	$\mathbf{w}^\top \mathbf{x}$	$\hat{y}$	Mistake	Update	New $\mathbf{w}$
1	A (-3, 1)	-1	0	none	yes	subtract A	(3, -1)
1	B (1, -2)	-1	5	+1	yes	subtract B	(2, 1)
1	C (-1, 3)	+1	1	+1	no	-	(2, 1)
1	D (-2, 3)	+1	-1	-1	yes	add D	(0, 4)
2	A (-3, 1)	-1	4	+1	yes	subtract A	(3, 3)
2	B (1, -2)	-1	-3	-1	no	-	(3, 3)
2	C (-1, 3)	+1	6	+1	no	-	(3, 3)
2	D (-2, 3)	+1	3	+1	no	-	(3, 3)
3	A (-3, 1)	-1	-6	-1	no	-	(3, 3)
3	B (1, -2)	-1	-3	-1	no	-	(3, 3)
3	C (-1, 3)	+1	6	+1	no	-	(3, 3)
3	D (-2, 3)	+1	3	+1	no	-	(3, 3)

Four mistakes; pass 3 makes none, so the rule stops at $\mathbf{w} = (3, 3)$, the line $x_2 = -x_1$. The update for D in pass 1 sent A back to the wrong side, and pass 2 corrected it.

Run 2: same points, same order, start from $\mathbf{w} = (-1, 1)$.

Pass	Example	y	$\mathbf{w}^\top \mathbf{x}$	$\hat{y}$	Mistake	Update	New $\mathbf{w}$
1	A (-3, 1)	-1	4	+1	yes	subtract A	(2, 0)
1	B (1, -2)	-1	2	+1	yes	subtract B	(1, 2)
1	C (-1, 3)	+1	5	+1	no	-	(1, 2)
1	D (-2, 3)	+1	4	+1	no	-	(1, 2)
2	A (-3, 1)	-1	-1	-1	no	-	(1, 2)
2	B (1, -2)	-1	-3	-1	no	-	(1, 2)
2	C (-1, 3)	+1	5	+1	no	-	(1, 2)
2	D (-2, 3)	+1	4	+1	no	-	(1, 2)

Two mistakes, and the rule stops at $\mathbf{w} = (1, 2)$, the line $x_2 = -x_1/2$. Both runs classify all four points correctly, with different lines (45° and 63° for $\mathbf{w}$): the rule stops at the first

line that works. Which line that is depends on the starting weights and on the order of the examples, as in the lecture's figure of three runs.

When the rule stops, and when it cannot

Rosenblatt [4] and Novikoff [2] proved that the rule always stops when it can. Suppose some line through the origin puts every example on its correct side with a gap, the *margin* γ : every example lies at least γ away from the line. Let R be the length $\|x\|$ of the longest example. Then, starting from $w = 0$, the perceptron makes at most $(R/\gamma)^2$ mistakes in total, whatever the order of the examples and the value of η . For the four points above, $R = \|D\| = \sqrt{13} \approx 3.61$, and the line perpendicular to $(0.6, 0.8)$ keeps every point at least $\gamma = 1$ away, so the bound is 13 mistakes. Run 1 made 4, and no order of the four examples makes more than 5.

One neuron cannot compute XOR. XOR labels four inputs: $(0, 1)$ and $(1, 0)$ are $+1$, and $(0, 0)$ and $(1, 1)$ are -1 . No line separates them, even with a bias: the two $+1$ points require $w_1 + b > 0$ and $w_2 + b > 0$, so $w_1 + w_2 + 2b > 0$, while the two -1 points require $b \leq 0$ and $w_1 + w_2 + b \leq 0$, so $w_1 + w_2 + 2b \leq 0$. The weights then cycle forever, and the mistakes per pass never reach zero. Later in the lecture, two hidden units draw two lines and an output unit combines them; training hidden units needs backpropagation.

The delta rule: the smooth version

The perceptron's rule is a heuristic, with no loss behind it. Widrow and Hoff [5] dropped the threshold and compared the label with the weighted sum $z = w^\top x$ itself. The squared error $\ell = \frac{1}{2}(y - z)^2$ is smooth, its gradient is $\partial\ell/\partial w = -(y - z)x$, and a step against the gradient is $\Delta w = \eta(y - z)x$. This is the perceptron's shape, learning rate $\times$ error $\times$ input, derived from a loss. The error is now graded, so every example moves the weights, by an amount that grows with how wrong the example is. The backpropagation notes apply the same shape, error $\times$ activity, to every weight of a network with hidden layers.

References

- [1] M. Minsky and S. Papert. *Perceptrons*. MIT Press, 1969.
- [2] A. B. J. Novikoff. On convergence proofs on perceptrons. In *Proceedings of the Symposium on the Mathematical Theory of Automata*, volume 12, pages 615–622. Polytechnic Institute of Brooklyn, 1962.
- [3] F. Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65:386–408, 1958. doi: 10.1037/h0042519.
- [4] F. Rosenblatt. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan Books, Washington, DC, 1962.
- [5] B. Widrow and M. E. Hoff. Adaptive switching circuits. In *IRE WESCON Convention Record, Part 4*, pages 96–104, 1960. doi: 10.21236/AD0241531.